

第五章

(第 3、4、5、6 (3)、8、9、11 题)

3. 假设一个 C 语言程序有两个源文件：main.c 和 test.c，内容如下图所示。

```
1  /* main.c */
2  int sum();
3
4  int a[4]={1, 2, 3, 4};
5  extern int val;
6  int main( )
7  {
8      val=sum();
9      return val;
10 }
```

```
1  /* test.c */
2  extern int a[];
3  int val=0;
4  int sum()
5  {
6      int i;
7      for (i=0; i<4; i++)
8          val += a[i];
9      return val;
10 }
```

题 3 用图

对于编译生成的可重定位目标文件 test.o，填写下表中各符号的情况，说明每个符号是否出现在 test.o 的符号表（.symtab 节）中，如果是的话，定义该符号的模块是 main.o 还是 test.o、该符号的类型是全局、外部还是本地符号？该符号出现在 test.o 中的哪个节（.text、.data 或.bss）？

符号	在 test.o 的符号表中？	定义模块	符号类型	节
a				
val				
sum				
i				

参考答案：

符号	在 test.o 的符号表中？	定义模块	符号类型	节
a	在	main.o	extern	.data
val	在	test.o	global	.data
sum	在	test.o	global	.text
i	不在	--	--	--

4. 假设一个 C 语言程序有两个源文件：main.c 和 swap.c，其中，main.c 的内容如图 5.9(a)所示，而 swap.c 的内容如下：

```
1  extern int buf[];
2  int *bufp0 = &buf[0];
3  static int *bufp1;
4
5  static void incr() {
6      static int count=0;
7      count++;
8  }
8
9  void swap() {
```

```

10    int temp;
11    incr();
12    bufp1=&bufp[1];
13    temp=*bufp0;
14    *bufp0=*bufp1;
15    *bufp1=temp;
16 }

```

对于编译生成的可重定位目标文件 `swap.o`，填写下表中各符号的情况，说明每个符号是否出现在 `swap.o` 的符号表（`.symtab` 节）中，如果是的话，定义该符号的模块是 `main.o` 还是 `swap.o`、该符号的类型是全局、外部还是本地符号？该符号出现在 `swap.o` 中的哪个节（`.text`、`.data` 或 `.bss`）？

符号	在 <code>swap.o</code> 的符号表中？	定义模块	符号类型	节
<code>buf</code>				
<code>bufp0</code>				
<code>bufp1</code>				
<code>incr</code>				
<code>count</code>				
<code>swap</code>				
<code>temp</code>				

参考答案：

符号	在 <code>swap.o</code> 的符号表中？	定义模块	符号类型	节
<code>buf</code>	在	<code>main.o</code>	<code>extern</code>	<code>.data</code>
<code>bufp0</code>	在	<code>swap.o</code>	<code>global</code>	<code>.data</code>
<code>bufp1</code>	在	<code>swap.o</code>	<code>local</code>	<code>.bss</code>
<code>incr</code>	在	<code>swap.o</code>	<code>local</code>	<code>.text</code>
<code>count</code>	在	<code>swap.o</code>	<code>local</code>	<code>.bss</code>
<code>swap</code>	在	<code>swap.o</code>	<code>global</code>	<code>.text</code>
<code>temp</code>	不在	--	--	--

5. 假设一个 C 语言程序有两个源文件：`main.c` 和 `proc1.c`，它们的内容如下图所示。

```

1  #include <stdio.h>
2  unsigned x=257;
3  short y, z=2;
4  void proc1(void);
5  void main()
6  {
7      proc1();
8      printf( "x=%u, z=%d\n" , x, z);
9      return 0;
10 }

```

a) `main.c` 文件

```

1  double x;
2
3  void proc1()
4  {
5      x=-1.5;
6  }

```

b) `proc1.c` 文件

题 5 用图

回答下列问题。

(1) 在上述两个文件中出现的符号哪些是强符号？哪些是 COMMON 符号？

- (2) 程序执行后打印的结果是什么？请分别画出执行第 7 行的 `proc1()` 函数调用前、后，在地址 `&x` 和 `&z` 中存放的内容。若 `main.c` 中第 3 行改为 “`short y=1, z=2;`”，打印结果是什么？
- (3) 修改文件 `proc1.c`，使得 `main.c` 能输出正确的结果（即 `x=257, z=2`）。要求修改时不能改变任何变量的数据类型和名字。

参考答案：

- (1) `main.c` 中强符号有 `x`、`z`、`main`，COMMON 符号有 `y`；`proc1.c` 中强符号有 `proc1`，COMMON 符号有 `x`。根据多重定义符号处理规则 2（若出现一次强符号定义和多次 COMMON 符号或弱符号定义，则按强符号定义为准），符号 `x` 的定义以 `main.c` 中的强符号 `x` 为准，即在 `main.o` 的 `.data` 节中分配 `x`，占 4 字节，随后是另一个强符号 `z` 占两个字节，`x` 和 `z` 都属于 `.data` 节，随后是 `.bss` 节，其中只有一个变量 `y`，按 4 字节对齐，因此，在 `z` 后面有两个字节空闲，再后面是变量 `y` 的空间。
- (2) 程序执行时，在调用 `proc1()` 函数之前，`&x` 中存放的是 `x` 的机器数 `0x0000 0101`，随后两个字节（地址为 `&z`）存放 `z` 的机器数 `0x0002`，再后面两个字节空闲。如下左图所示。

	0	1	2	3
<code>&z</code>	02	00	---	---
<code>&x</code>	01	01	00	00

	0	1	2	3
<code>&z</code>	00	00	F8	BF
<code>&x</code>	00	00	00	00

在调用 `proc1()` 函数后，因为 `proc1()` 中的符号 `x` 是 COMMON 符号，因此，`x` 的定义以 `main` 中的强符号 `x` 为准，执行 `x=-1.5` 后，便将 “-1.5” 的机器数 `BFF80000 00000000H` 存放到了 `&x` 开始的 8 个字节中。即 `&x` 中为其低 32 位的 `00000000H`，`&z` 中为高 32 位的 `BFF80000H` 中的低 16 位 `0000H`，`z` 后面的两个空闲字节中为高 16 位 `BFF8H`。如上右图所示。

因此，最终打印的结果如下：`x=0, z=0`。

若 `main.c` 中第 3 行改为 “`short y=1, z=2;`”，则 `x`、`y`、`z` 都是强符号，都被分配在 `.data` 节中，因此，`x` 占 4 个字节，随后是 `y` 占两个字节，`z` 占两个字节，`proc1()` 函数执行前后的存储内容如下图所示，因此，最终打印的结果如下：`x=0, z=-16392`。

	0	1	2	3
<code>&y</code>	01	00	02	00
<code>&x</code>	01	01	00	00

	0	1	2	3
<code>&y</code>	00	00	F8	BF
<code>&x</code>	00	00	00	00

- (3) 只要将文件 `proc1.c` 中的第 1 行修改为 “`static double x;`” 就可以使得 `proc1` 中的 `x` 设定为本地变量，从而在 `proc1.o` 的 `.data` 节中专门分配存放 `x` 的 8 个字节空间，而不会和 `main` 中的 `x` 共用同一个存储地址。因此，也就不会破坏 `main` 中 `x` 和 `z` 的值。

6. 以下每一小题给出了两个源程序文件，它们被分别编译生成可重定位目标模块 `m1.o` 和 `m2.o`。在模块 `mj` 中对符号 `x` 的任意引用与模块 `mi` 中定义的符号 `x` 关联记为 `REF(mj.x)→DEF(mi.x)`。请在下列空格处填写模块名和符号名以说明给出的引用符号所关联的定义符号，若发生链接错误则说明其原因；若从多个定义符号中任选则给出全部可能的定义符号，若是局部变量则说明不存在关联。

(3) `/* m1.c */`
`int p1(void);`
`int p1;`
`int main()`
`{`

`/* m2.c */`
`int x=10;`
`int main;`
`int p1()`
`{`

```

int x=p1();
return x;
}

main=1;
return x;
}

```

- ① REF(m1.main)→DEF(_____._____)
- ② REF(m2.main)→DEF(_____._____)
- ③ REF(m1.p1)→DEF(_____._____)
- ④ REF(m1.x)→DEF(_____._____)
- ⑤ REF(m2.x)→DEF(_____._____)

参考答案:

(3) main 在 m1 中是强符号，在 m2 中是 COMMON 符号，因此链接器选择强定义

- ① REF(m1.main)→在 m1 中不存在对 main 的引用
- ② REF(m2.main)→DEF(m1.main)
- ③ REF(m1.p1)→DEF(m2.p1)
- ④ REF(m1.x)→在 m1 中引用的 x 是局部变量，不存在关联
- ⑤ REF(m2.x)→DEF(m2.x)

8. 下图中给出了用 OBJDUMP 显示的某个可执行目标文件的程序头表（段头表）的部分信息，其中，可读写数据段（Read/write data segment）的信息表明，该数据段对应虚拟存储空间中起始地址为 0x8049448、长度为 0x104 个字节的存储区，其数据来自可执行文件中偏移地址 0x448 开始的 0xe8 个字节。这里，可执行目标文件中的数据长度和虚拟地址空间中的存储区大小之间相差了 28 字节。请解释可能的原因。

```

Read-only code segment
LOAD off 0x00000000 vaddr 0x08048000 paddr 0x08048000 align 2**12
      filesz 0x00000448 memsz 0x00000448 flags r-x

Read/write data segment
LOAD off 0x00000448 vaddr 0x08049448 paddr 0x08049448 align 2**12
      filesz 0x000000e8 memsz 0x00000104 flags rw-

```

某可执行目标文件程序头表的部分内容

参考答案:

在可执行目标文件中描述的“可读写数据段”由所有可重定位目标文件中的.data 节合并生成的.data 节、所有可重定位目标文件中的.bss 节合并生成的.bss 节这两部分组成。.data 节由已初始化且初值不为 0 的全局变量和静态变量组成，因而其初始值必须记录在可执行文件中，而.bss 节由未初始化或初始化为 0 的全局变量和静态变量组成，因而在可执行目标文件中无需记录其值，只要描述总的长度和每个变量的起始位置即可。

根据上图中的内容可知，.data 节中已初始化且初值不为 0 的全局变量和静态变量的数据长度为 0xe8。因此，虚拟地址空间中长度为 0x104 字节的可读写数据段中，开始的 0xe8 个字节取自.data 节，后面的 28 字节对应.bss 节中数据。

9. 假定 a 和 b 是可重定位目标文件或静态库文件，a→b 表示 b 中定义了一个被 a 引用的符号。对于以下每一小题出现的情况，给出一个最短命令行（含有最少数量的可重定位目标文件或静态库文件参数），使

得链接器能够解析所有的符号引用。

- (1) p.o→libx.a→liby.a→p.o
- (2) p.o→libx.a→liby.a 同时 liby.a→libx.a
- (3) p.o→libx.a→liby.a→libz.a 同时 liby.a→libx.a→libz.a

参考答案:

- (1) gcc -static -o p p.o libx.a liby.a p.o
- (2) gcc -static -o p p.o libx.a liby.a libx.a
- (3) gcc -static -o p p.o libx.a liby.a libx.a libz.a

11. 图 5.20 给出了图 5.9(b)

中 swap 源代码对应的 swap.o 文件中 .text 节和 .rel.text 节的内容, 图中显示 .text 节共有 6 处需重定位。假定链接后生成的可执行文件中 buf 和 bufp0 的存储地址分别是 0x80495c8 和 0x80495d0, bufp1 的存储地址位于 .bss 节的开始, 为 0x8049620。根据对图 5.20 的分析, 仿照例子填写下表, 以指出各个重定位的符号名、相对于 .text 节起始位置的位移、所在指令行号、重定位类型、重定位前的内容、重定位后的内容。

1	Disassembly of section .text:					
2	00000000 <swap>:					
3	0:55		push	%ebp		
4	1:89 e5		mov	%esp,%ebp		
5	3:83 ec 10		sub	\$0x10,%esp		
6	6:c7 05 00 00 00 00 04		movl	\$0x4,0x0		
7	d:00 00 00					
8	8: R_386_32	.bss				
9	c: R_386_32	buf				
10	10:a1 00 00 00 00		mov	0x0,%eax		
11	11: R_386_32	bufp0				
12	15:8b 00		mov	(%eax),%eax		
13	17:89 45 fc		mov	%eax,-0x4(%ebp)		
14	1a:a1 00 00 00 00		mov	0x0,%eax		
15	1b: R_386_32	bufp0				
16	1f: 8b 15 00 00 00 00		mov	0x0,%edx		
17	21: R_386_32	.bss				
18	25:8b 12		mov	(%edx),%edx		
19	27:89 10		mov	%edx,(%eax)		
20	29:a1 00 00 00 00		mov	0x0,%eax		
21	2a: R_386_32	.bss				
22	2e:8b 55 fc		mov	-0x4(%ebp),%edx		
23	31:89 10		mov	%edx,(%eax)		
24	33:c9		leave			
25	34:c3		ret			

图 5.20 swap.o 中 .text 节和 .rel.text 节内容

序号	符号	位移	指令所在行号	重定位类型	重定位前内容	重定位后内容
1	bufp1 (.bss)	0x8	6~7	R_386_32	0x00000000	0x8049620
2						
3						
4						
5						
6						

参考答案:

序号	符号	位移	指令所在行号	重定位类型	重定位前内容	重定位后内容
1	bufp1 (.bss)	0x8	6~7	R_386_32	0x00000000	0x8049620
2	buf (.data)	0xc	6~7	R_386_32	0x00000004	0x80495cc

3	bufp0 (.data)	0x11	10	R_386_32	0x00000000	0x80495d0
4	bufp0 (.data)	0x1b	14	R_386_32	0x00000000	0x80495d0
5	bufp1 (.bss)	0x21	17	R_386_32	0x00000000	0x8049620
6	bufp1 (.bss)	0x2a	21	R_386_32	0x00000000	0x8049620